

01 SQL Retrieval Test Set - Working Plan

This is a collaborative planning document for a retrieval-focused test set for supervised agents. Each test represents a user request and evaluates whether the agent can retrieve the correct information, use the appropriate tools when needed, and avoid inventing information that is not available.

The final tests may involve a single model request, one or more tool calls, or multiple dependent retrieval steps. The exact SQL/SOQL dialect, data model, and fixture records still need to be confirmed before the YAML tests are written.

Test-set objectives

- Verify that the agent identifies the right record or records from natural-language requests.
- Verify that the agent selects the correct fields and returns the requested information accurately.
- Verify that the agent can perform retrieval without tools when the answer is already available in context.
- Verify that the agent uses retrieval tools when the answer depends on external or Salesforce data.
- Verify that the agent can recover from an invalid query, incomplete result, or failed first attempt.
- Verify that the agent handles ambiguity, missing records, empty results, and multiple matches safely.
- Verify that multi-step retrieval produces a grounded final answer based on the records actually found.
- Measure both retrieval correctness and unnecessary or incorrect tool use.

Candidate test titles and descriptions

Direct and simple retrieval

- **01-sql-exact-record-lookup** - Retrieve one record using an exact identifying value, such as an account name or record number, and return the requested fields.
- **01-sql-multi-field-retrieval** - Retrieve a single record with a slightly fuzzy search and return several requested fields without omitting or substituting values.
- **01-sql-context-only-answer** - Answer from information already present in the conversation or supplied context, testing that the agent does not make an unnecessary tool call.
- **01-sql-required-retrieval** - Recognize that the answer is not present in context and use the available retrieval tool before answering.

Natural-language matching

- **01-sql-case-and-whitespace-variation** - Match a record when the user's capitalization, spacing, or punctuation differs from the stored value.
- **01-sql-partial-name-search** - Locate a record from a partial or abbreviated name while returning the correct full record.

- **01-sql-alias-or-common-name** - Resolve a user-facing alias or common business name to the appropriate stored record.
- **01-sql-natural-language-filter** - Translate a plain-language condition into the correct query filter and return only matching records.
- **01-sql-compound-filter** - Apply multiple conditions together, such as region plus status or industry plus employee count.
- **01-sql-or-filter** - Correctly handle alternatives in the request, such as records matching one of several regions or statuses.

Result cardinality and ambiguity

- **01-sql-no-match** - Handle a query that returns no records, clearly report that no match was found, and avoid fabricating an answer.
- **01-sql-multiple-matches** - Handle several matching records and either return the relevant set or ask a focused clarification question when one record is required.
- **01-sql-ambiguous-identifier** - Recognize that the user's identifier is ambiguous and use additional fields or clarification rather than selecting arbitrarily.
- **01-sql-duplicate-looking-records** - Distinguish between records with similar names using a secondary attribute such as location, type, or account number.
- **01-sql-null-field** - Retrieve a valid record whose requested field is empty and report the missing value accurately.

Numeric, date, and ordered retrieval

- **01-sql-numeric-threshold** - Retrieve records above, below, or within a numeric threshold, such as employee count, revenue, or score.
- **01-sql-date-range** - Retrieve records created, updated, or otherwise dated within a natural-language time range.
- **01-sql-relative-date** - Interpret a request such as "this quarter," "last month," or "in the past 30 days" using the appropriate date logic.
- **01-sql-sort-and-top-n** - Sort matching records by a requested field and return the top or bottom N results.
- **01-sql-count-results** - Return the count of records satisfying the user's conditions rather than returning the records themselves.
- **01-sql-aggregate-results** - Compute an aggregate such as sum, average, minimum, or maximum over the matching records.

Relationships and multi-step retrieval

- **01-sql-related-record-lookup** - Retrieve a record and then fetch related records, such as an account and its contacts or opportunities.
- **01-sql-dependent-lookup** - Use the result of a first query as the input to a second query, testing dependent tool calls and correct propagation of identifiers.
- **01-sql-multi-record-dependent-lookup** - Retrieve several parent records and then gather related child records for each one.
- **01-sql-join-or-relationship-filter** - Filter or report records based on a field belonging to a related object.
- **01-sql-schema-discovery-before-query** - Inspect the available object and field schema before querying when the field names are not obvious.
- **01-sql-field-name-correction** - Recover when an initial query uses an invalid field name by inspecting schema or interpreting the tool error.

Tool-use reliability and recovery

- **01-sql-no-tool-when-not-needed** - Complete a request from supplied data without invoking retrieval tools, testing restraint and context use.
- **01-sql-one-tool-call-suffices** - Stop after a successful retrieval when the requested answer is complete, rather than making redundant calls.
- **01-sql-retry-after-query-error** - Correct and retry a malformed or rejected query, then return the answer from the successful result.
- **01-sql-retry-after-incomplete-result** - Recognize that the first result is insufficient and issue a narrower or additional query to obtain the requested information.
- **01-sql-tool-result-grounding** - Ensure every returned value is supported by the tool result and that values from unrelated records are not mixed together.
- **01-sql-tool-error-reporting** - Handle an unavailable or failed retrieval tool transparently, without presenting an unverified answer as fact.

Response interpretation and safety

- **01-sql-explicit-field-mapping** - Return values mapped to the correct user-requested labels, testing that similarly named fields are not confused.
- **01-sql-record-order-preservation** - Preserve or explicitly apply the requested ordering when returning multiple records.
- **01-sql-limited-result-set** - Handle a result limit or truncated response by narrowing the query, paginating, or clearly explaining the limitation.
- **01-sql-sensitive-field-restraint** - Return only the fields requested by the user and avoid exposing unrelated sensitive fields present in the record.
- **01-sql-unsupported-request** - Explain when the requested information cannot be retrieved from the available objects or tools instead of guessing.
- **01-sql-answer-format** - Return retrieved results in the requested format, such as a value, short list, table, or grouped summary, while preserving factual accuracy.

Decisions to make before authoring YAML

- Confirm whether “SQL” means standard SQL, Salesforce SOQL, or a retrieval abstraction exposed through tools.
- Define the fixture objects, fields, and stable records used by the tests.
- Decide which tests are context-only, tool-required, tool-optional, or explicitly multi-step.
- Decide whether clarification questions are allowed outcomes or whether every test must be answerable in one run.
- Decide whether tool-use efficiency is evaluated separately from answer correctness.
- Define the expected response shape for each test so the normalizer schema and evaluator checks can be generated consistently.
- Decide whether tests with live or mutable data should use exact expected values, relational assertions, or controlled fixtures.
- Confirm the naming convention for test files, test IDs, categories, and the test-set filename.